NIALL CASSIDY

PROTOTYPE

A2 WJEC COMPUTER SCIENCE

NIALL CASSIDYNDIDATE NO: 0109
CENTER NO: 71741

Section 4: Prototype (10/100) marks

- **Justified the choice of areas to be included in the prototype system and explained the reasons for omitting the remaining areas**
- **Created a comprehensive range of screens and outputs for the chosen areas of the solution**
- **Created a functioning system that carries out all chosen processes using realistic data and fulfils all requirements for data output and storage**
- **Evaluated the functioning of the prototype and justified the good features of the prototype solution**
- **Described the shortcomings and made specific suggestions for improvement.**

4.0 Prototype

The aim of the prototype was to develop a semi-functional representation of the system to be designed for Mulligan Motors, focusing specifically on the manager-side functionalities. I choose to focus on the managers specification for two main reasons: Firstly, the nature of a managers duties requires it to have a broad access to a majority of the overall system. Secondly, this is one of the more important stakeholders in the system, I believe making the prototype on the managerial side of the system will help me improve my skills for the finished project. To ensure the prototype's success and effectiveness, it was essential to include sufficient features and areas of the managerial system. This would enable the end user to gain a comprehensive understanding of how the final solution is expected to operate.

4.1 Features to be included

1. **Login:** The login functionality is included to simulate secure access to the system. Although basic, it demonstrates the potential for future role-based access. It allows the prototype to maintain a professional appearance while enabling the manager, the primary user in this stage, to log in. Since the manager requires broad access to the entire system, this feature ensures the prototype reflects real-world requirements effectively.

2. **Manager's Home Screen:** The home screen serves as a central navigation hub, providing easy access to the different parts of the system, including managing customers, vehicles, and orders. Since the manager requires broad access to the whole system, the home screen showcases the full scope of the prototype in an organized and intuitive manner, making it an excellent way to demonstrate the system's functionality.

3. **Database Tables:** The inclusion of database tables for customers, vehicles, and orders lays the groundwork for system functionality. These tables are critical for handling structured business data. The focus is on maintaining data integrity and accuracy rather than real-time data manipulation, ensuring that the prototype demonstrates robust data handling and storage capabilities.

4. **Data Manipulation Facilities (View/Edit/Add/Delete Records):** The ability to view, edit, add, and delete records highlights the prototype's practicality for managing essential data. While real-time collaboration is not emphasized, these facilities showcase how the system ensures data integrity

and accuracy by allowing the manager to maintain and correct records as needed. This functionality aligns with the managerial need for full control over system data.

5. **Search Function:** The search function is included to allow quick and efficient retrieval of specific records based on IDs. This feature demonstrates the system's ability to handle and locate data accurately, which is a critical aspect of data integrity. It also reflects the importance of an efficient workflow for managerial tasks, emphasizing the practicality of the prototype.

4.2 Explanation of Features to be Omitted in the Prototype

1. **Data Validation:** While important for maintaining data integrity, data validation is omitted in this prototype to keep the focus on demonstrating the system's core functionality. Validation can be integrated in future development stages once the foundational data handling mechanisms are finalized.

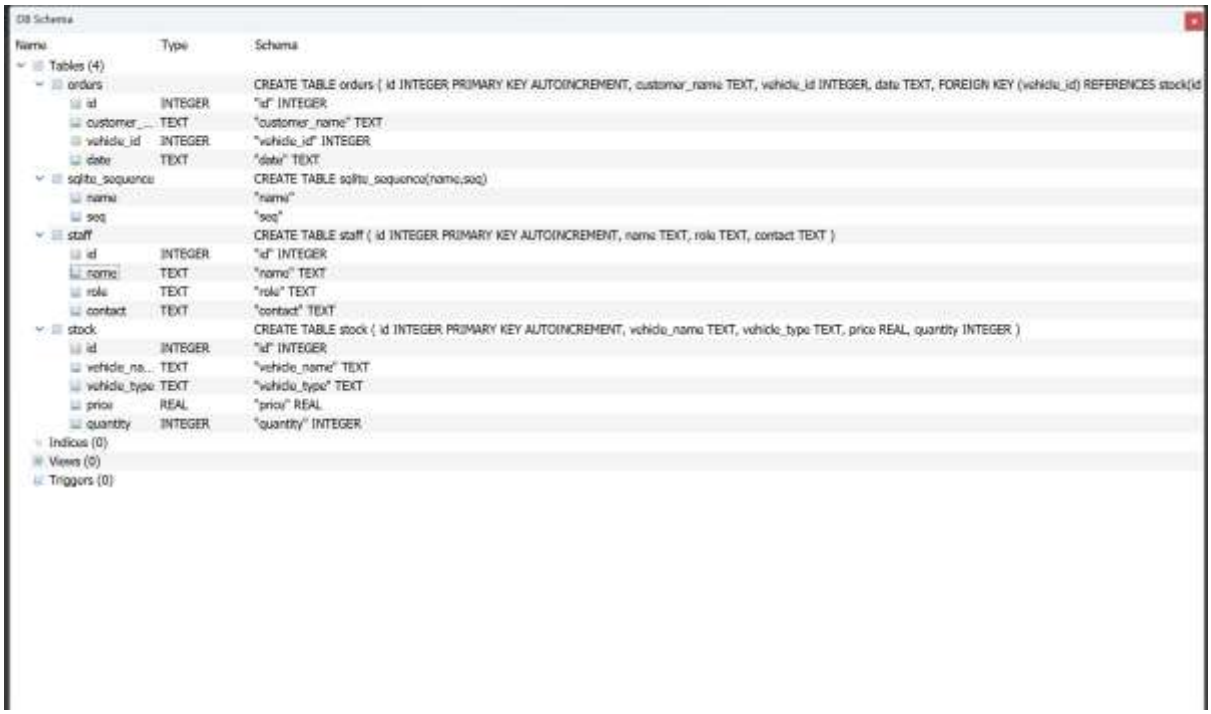
2. **Access Permissions:** Complex role-based access permissions have been excluded as the prototype is designed primarily for managerial use. Since the manager requires broad access to the whole system, role-specific restrictions are unnecessary at this stage and can be implemented later.

3. **Staff Views:** A staff view has been omitted to keep the prototype focused on the manager's responsibilities. Since the manager oversees all system functions, including a staff-specific interface is not critical for showcasing the core features of the system at this stage

4. **Booking Test Drives:** Test drive bookings have been excluded to prioritize the management of customers, vehicles, and orders. Including this feature would divert focus from the core data manipulation aspects, which are more critical to demonstrating the prototype's capabilities. Whilst test-drives are one of more important features, I feel that it may be too advanced to do in a simple prototype built to allow me to quickly showcase and get a feel for the project.

5. **Output Reports:** Generating reports is omitted to focus on interactive data handling and manipulation. While reporting is a useful feature, it is a secondary feature as it focuses on the aesthetic of the software. In this prototype i decided to prioritise that the prototype demonstrates strong data integrity and an intuitive workflow for managing information.

4.3 Creating the database



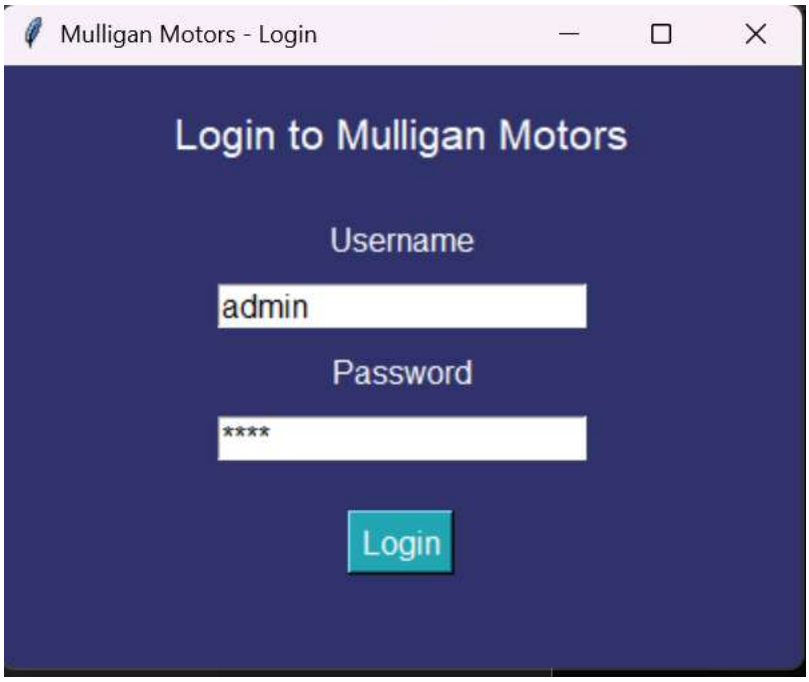
Below is the schema for the relational database created to store the data used within the Mulligan Motors management application. The **Customer** table is related to the **Orders** table to maintain a record of every order placed by each customer. The **Orders** table is also linked to the **VehicleStock** table, which stores information about the vehicles, including their price, type, and available quantity. This structure ensures that all customer orders are connected to the specific vehicles in stock, allowing for efficient tracking and management of inventory and sales within the system. It should be noted this is not the schema of the full system, as there are still some features missing from the prototype, such as payslips.

4.4 Graphical User Interfaces

During the discussion stage of the project, the owner of Mulligan Motors clearly stated he would like a blue colour scheme. Resultingly, I have made the background colour of the prototype this colour to reflect what the finished system will look like. For all text in the program, I used the font called ‘Arial’ at size 12. This font delivers an easy to read yet professional look to the system.

Login

- The login screen ensures secure access to the system by requiring a username and password. Upon successful login, the manager is directed to the main menu. This prototype limits the login page, however, as the access table is not included. As a result, there is only one set of login details(Username: admin Password: 1234.)

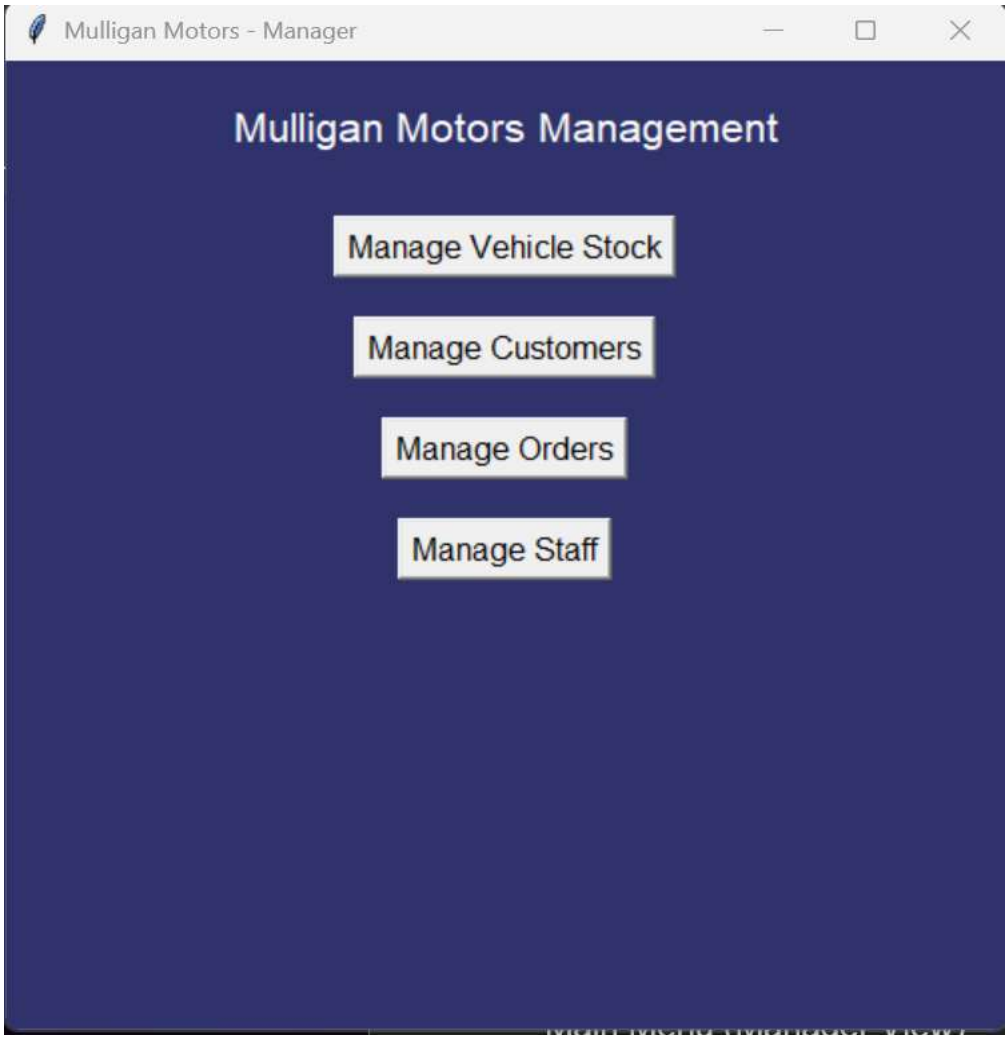


Features:

- **Username and Password Fields**: Allows the user to input their login credentials.
- **Password Field**: Replaces user’s inputs with asterisks*
- **Login Button**: Verifies the entered credentials and grants access to the main menu if correct.

Main Menu (Manager View)

- This screen serves as the central hub, allowing the manager to navigate through various sections of the application. Each button leads to a specific feature, making the interface intuitive and user-friendly. Not all functions are currently available, such as payslips and test drives, so this menu will be expanded in the final version.

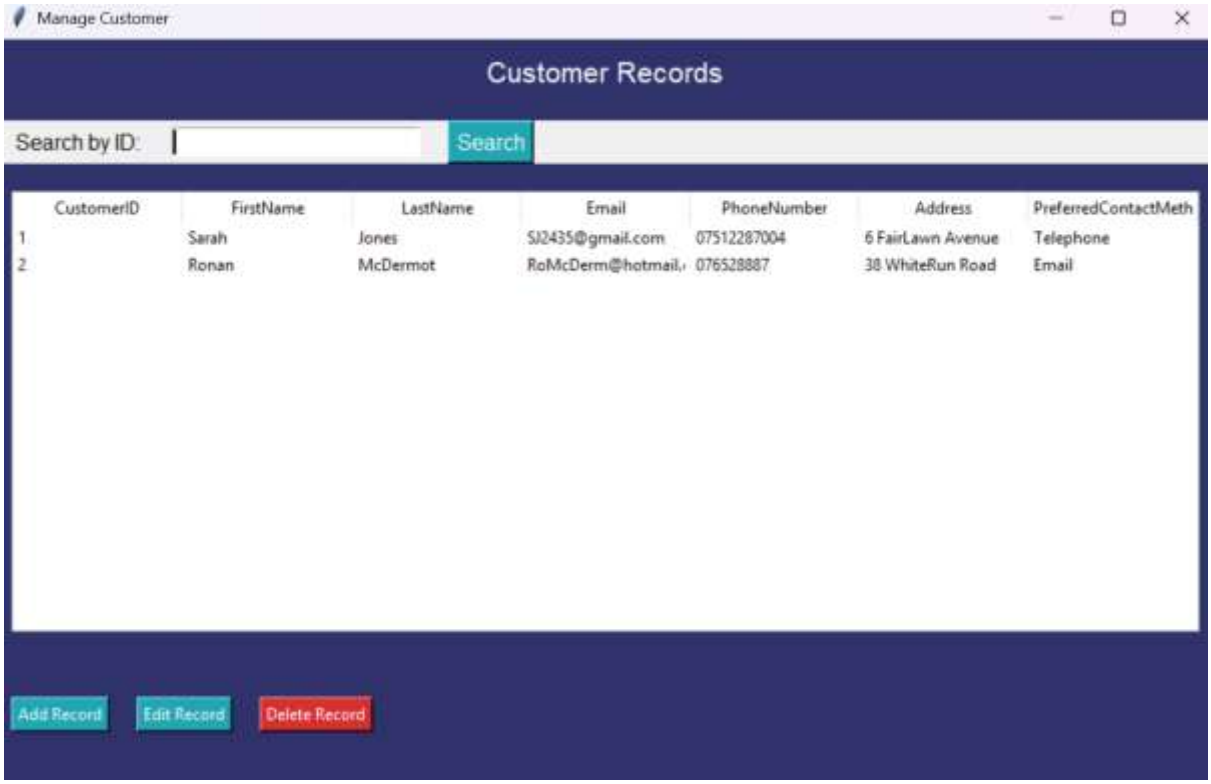


Features:

- **Navigation Buttons**: Options like "Manage Customers," "Manage Vehicle Stock," "Manage Staff," and "Manage Orders."
- Clearly labelled window informing the user this is the manager's menu.

Customer Table

- The customer table provides an overview of all customer data, enabling the manager to quickly view or search for a customer's details.

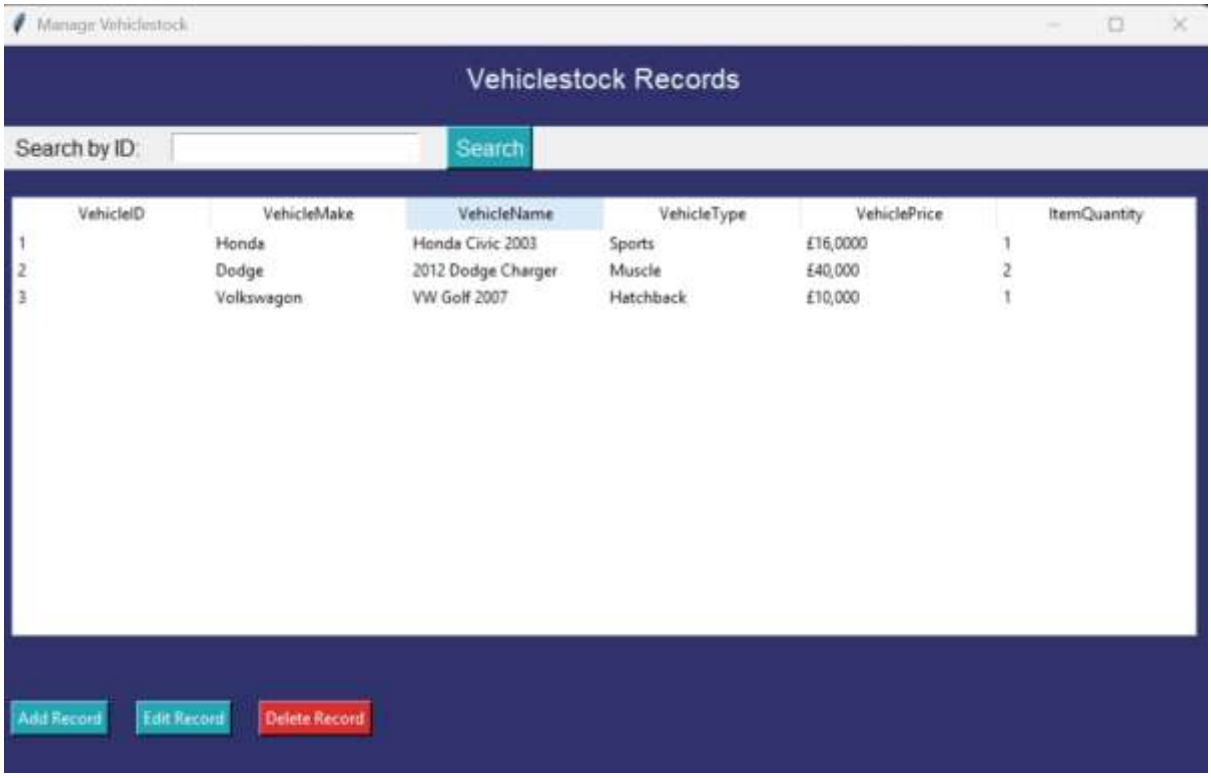


Features:

- **Table View**: Displays all customer records in a structured format using the Treeview widget.
- **Search Functionality**: Allows the manager to find a specific customer using their unique ID.
- **Add/Delete/Edit records**: These buttons are appropriately colour coded to ensure no accidental deletions.

Vehicle Stock

- This section helps the manager monitor and manage inventory, ensuring vehicle stock is up-to-date.



Features:

- **Table View**: Displays all vehicle stock details, including make, model, price, type, and quantity.
- **Search Functionality**: Allows the manager to locate specific vehicles by their unique ID.
- **Add/Delete/Edit records**: These buttons are appropriately colour coded to ensure no accidental deletions.

Staff

- The staff section assists the manager in maintaining records of employees and their roles within the company.

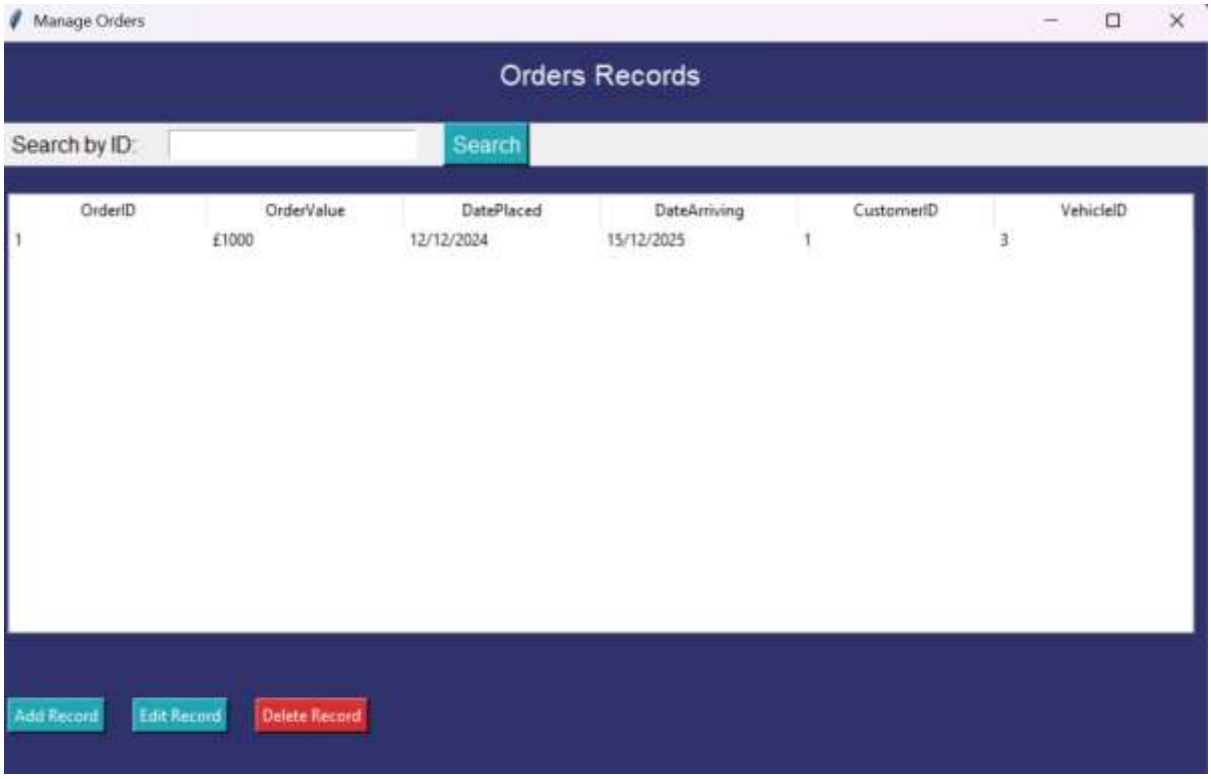


Features:

- **Table View**: Displays staff details such as name, contact information, role, and title.
- **Search Functionality**: Enables the manager to find specific staff members using their ID.
- **Add/Delete/Edit records**: These buttons are appropriately colour coded to ensure no accidental deletions.

Orders

- This section tracks customer orders and displays them, providing a comprehensive view of transactions and associated details.



Features:

- **Table View**: Displays order details, including customer ID, vehicle ID, order date, and total value.
- **Relational links:** The foreign keys (CustomerID, VehicleID) create a relationship between this table and the customers and vehicle stock table. This improves data integrity in the system.
- **Search Functionality**: Lets the manager view specific orders by their unique ID.

4.5 Adding data

- Adding data allows the manager to expand the database with new customers, vehicles, orders, or staff information. The input fields ensure all necessary data is entered correctly.

```
def add_record():
    def save_record():
        conn = sqlite3.connect('mulligan_motors.db')
        cursor = conn.cursor()
        values = [entry.get() for entry in entries]
        cursor.execute(insert_query, values)
        conn.commit()
        conn.close()
        add_window.destroy()
        load_data()

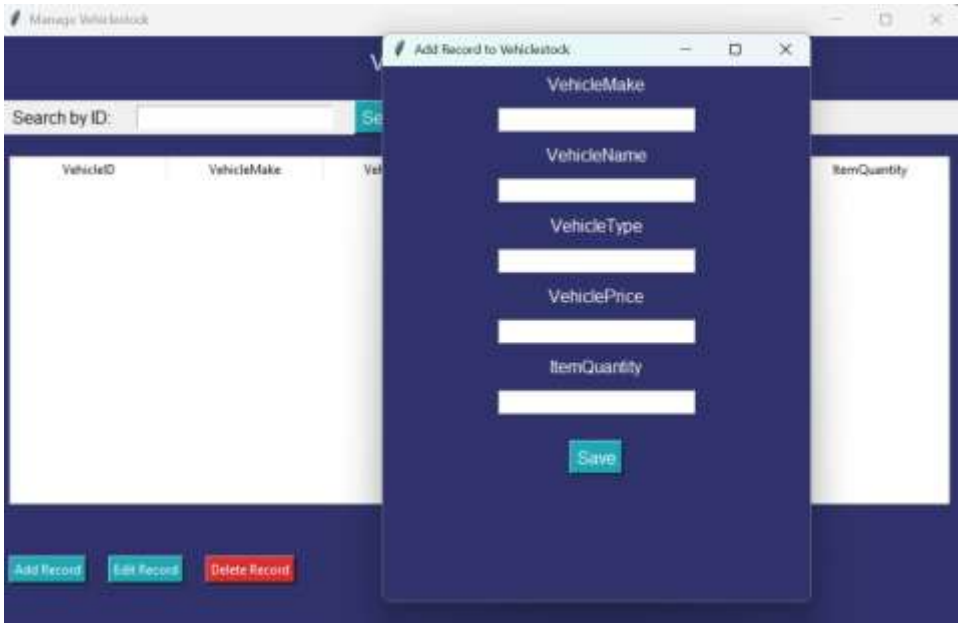
    add_window = Toplevel(table_window)
    add_window.title(f"Add Record to {table_name.title()}")
    add_window.geometry("400x500")
    add_window.config(bg="#303360")

    entries = []
    for field in fields[1:]:  # Skip ID
        Label(add_window, text=field, font=("Arial", 12), bg="#303360", fg="white").pack(pady=5)
        entry = Entry(add_window, font=("Arial", 12))
        entry.pack(pady=5)
        entries.append(entry)

    Button(add_window, text="Save", command=save_record, font=("Arial", 12), bg="#22a605", fg="white").pack(pady=20)
```

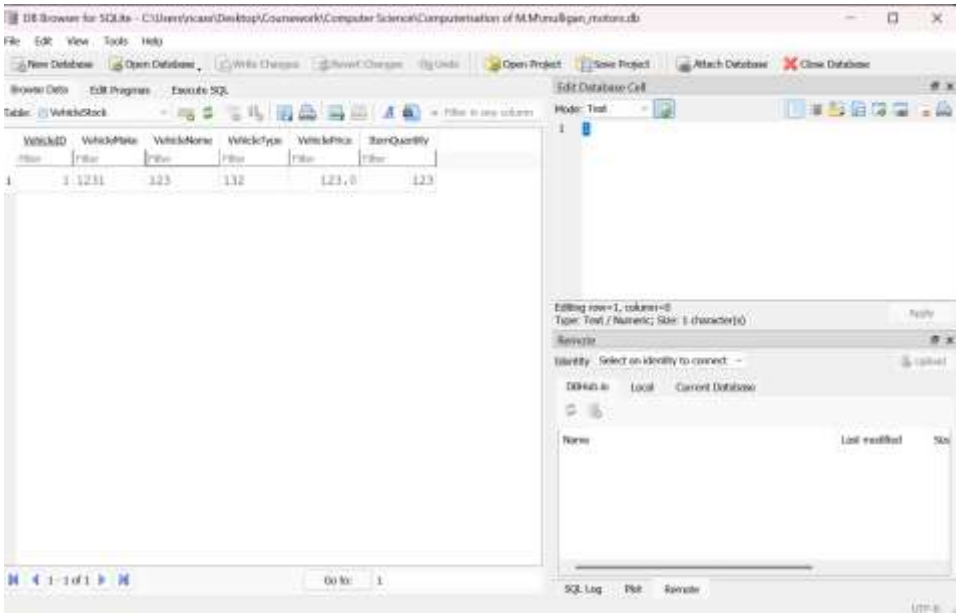
Features:

- **Add Record Button**: Opens a form where the manager can input details to create a new record.
- **Save Button:** Saves the new record to the corresponding table.



A new window opens with the template to input the new data.

The option to manually enter the records ID is not present. This is as the system will automatically set and increment the unique identifier of a record.



The test data inputted into the prototype software has saved and written the data into the database file, which may be retrieved later.

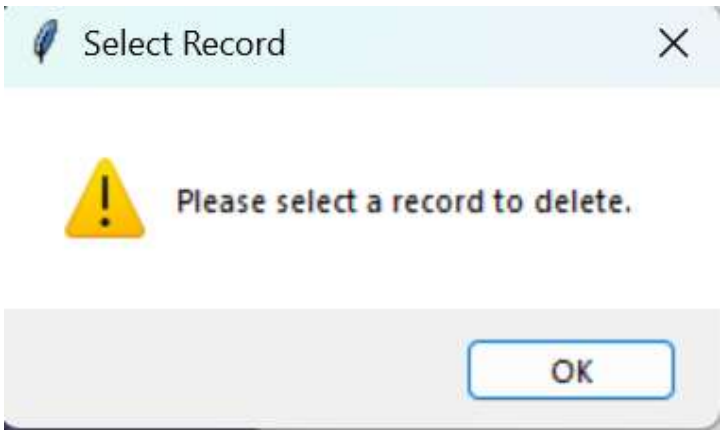
4.6 Deleting Data

- Deleting data is crucial for maintaining a clean and accurate database. The confirmation prompt minimizes errors by giving the manager a chance to reconsider the deletion.

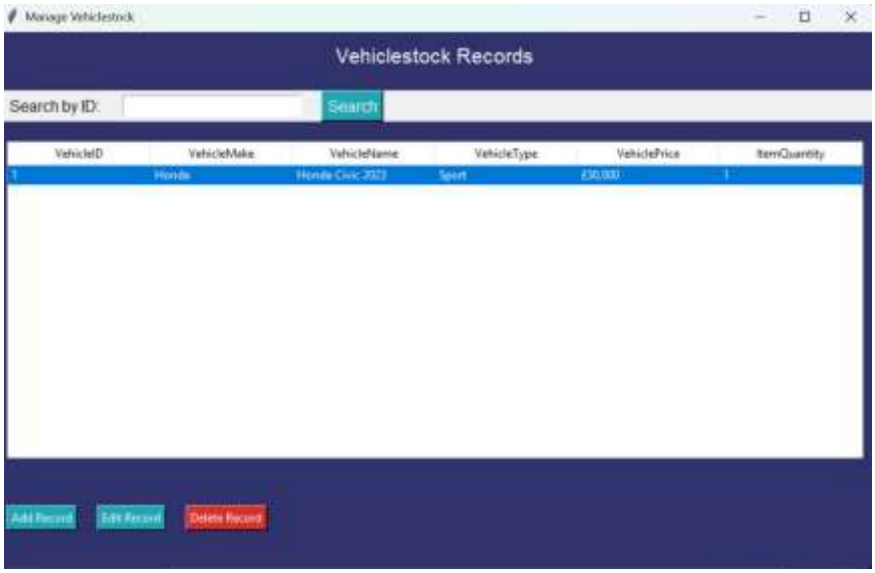
```
153         def delete_record():
154             selected = table_tree.focus()
155             if not selected:
156                 messagebox.showwarning("Select Record", "Please select a record to delete.")
157                 return
158             record_id = table_tree.item(selected)['values'][0]
159             conn = sqlite3.connect('mulligan_motors.db')
160             cursor = conn.cursor()
161             cursor.execute(f"DELETE FROM {table_name} WHERE {fields[0]} = ?", (record_id,))
162             conn.commit()
163             conn.close()
164             load_data()
```

Features:

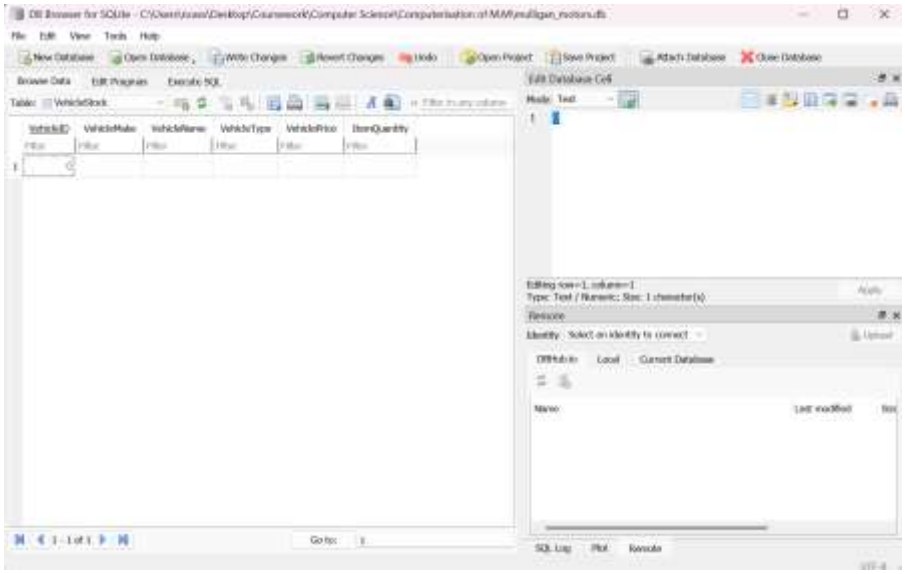
- **Delete Button**: Removes the selected record from the table. This is coloured red to emphasize its potential dangers if used haphazardly
- **Confirmation Prompt**: Ensures accidental deletions are avoided by asking for confirmation before proceeding.



If no record is selected a message box will appear.



The record to be deleted must be highlighted in blue for the system to erase it. This is done by clicking on the record.



The record to be deleted was successfully removed from the database file.

4.7 Editing Data

- Editing allows the manager to correct or update information, ensuring that all data in the system remains accurate and relevant. This was achieved using the Add/Delete functions previously mentioned. This will save space in the system, and make it easier to maintain.

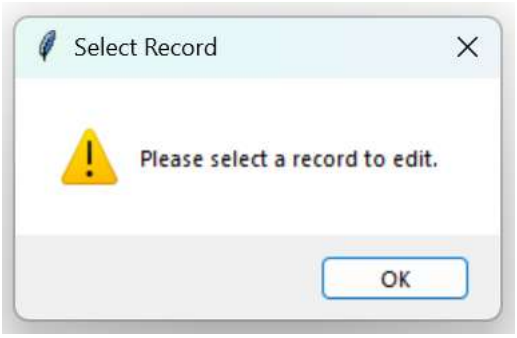


```
def edit_record():
    selected = table_tree.focus()
    if not selected:
        messagebox.showwarning("Select Record", "Please select a record to edit.")
        return

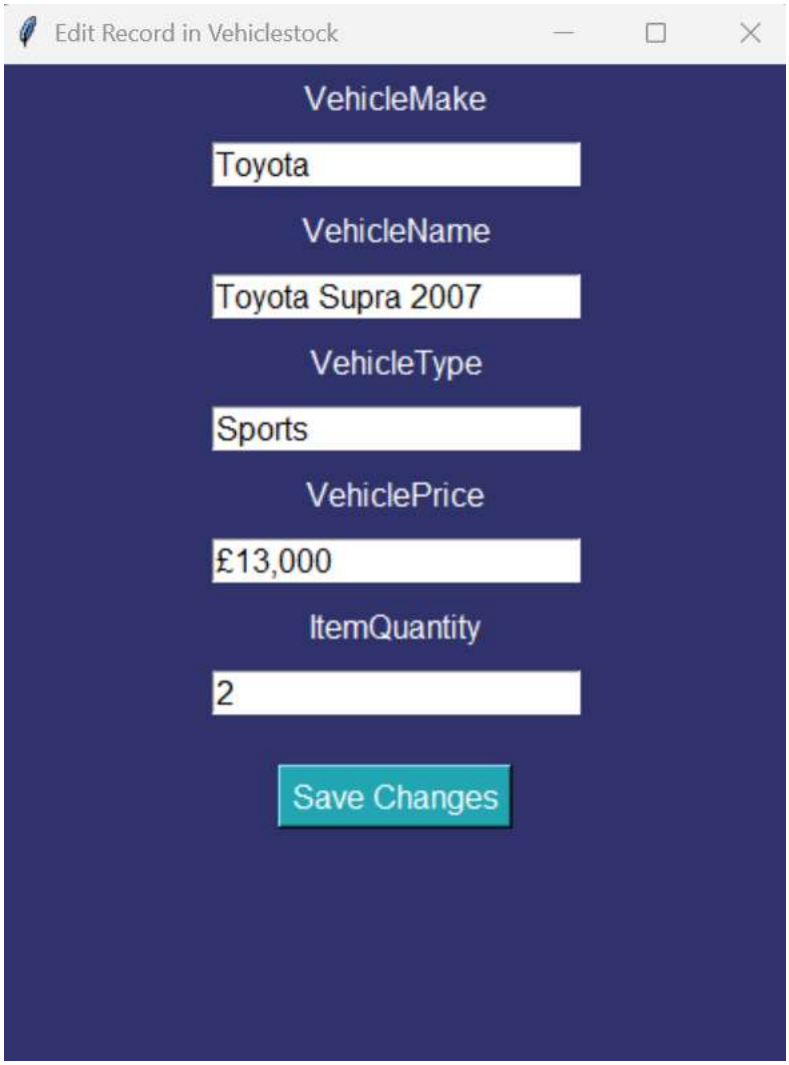
    record = table_tree.item(selected)['values']
    delete_record()  # Delete the selected record
    add_record(record[1:])  # Open the add record window with pre-filled values
```

Features:

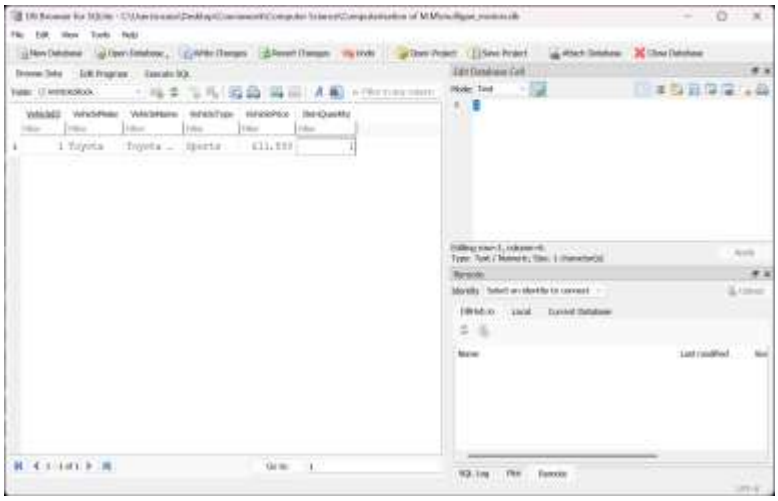
- **Edit Record Button**: Opens a form populated with the selected record's details.
- **Save Changes Button**: Updates the record with the new data entered by the manager.



If no record is selected an error message box will appear



A pre-filled form will appear when the edit button is clicked. The user can make any changes and then press save. The old record will be replaced with this one.



Using the edit feature, I was able to, for example, mark down the price of a vehicle. This was saved in the database as seen in the screenshot. This feature would be useful for fixing mistakes.

4.8 Searching Data

- Searching improves efficiency by helping the manager locate specific records without having to manually browse through all data.

```
def search_by_id():
    search_id = search_entry.get()
    if not search_id.isdigit():
        messagebox.showwarning("Invalid Input", "Please enter a valid numeric ID.")
        return

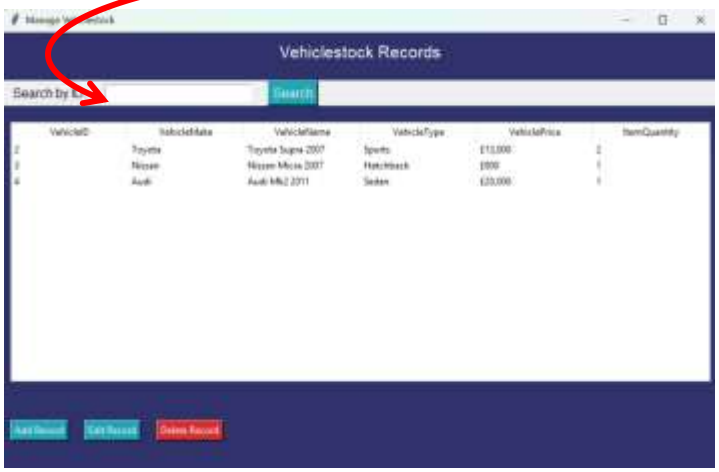
    conn = sqlite3.connect('mulligan_motors.db')
    cursor = conn.cursor()
    cursor.execute(f"SELECT * FROM {table_name} WHERE {fields[0]} = ?", (search_id,))
    record = cursor.fetchone()
    conn.close()

    for item in table_tree.get_children():
        table_tree.delete(item)

    if record:
        table_tree.insert("", "end", values=record)
    else:
        messagebox.showinfo("No Record Found", f"No record found with ID {search_id}.")
```

Features:

- **Search Bar**: Accepts input for searching specific records by unique ID.
- **Search Results**: Displays only the matching records in the table view.



If Mulligan Motors wishes to search for a certain record, they may enter the ID in the intuitive search bar.



After inputting an ID of '3' into the search bar, the corresponding record shows up

4.9 Evaluation & Areas of Improvement

I believe the Mulligan Motors prototype effectively demonstrates the core functionalities required for managing customers, vehicles, orders, and staff. It features a simple yet functional interface with intuitive navigation, which I intentionally designed to be accessible to users with basic technical skills. The relational database design, including foreign keys, ensures data consistency and integrity across the system. Key functionalities such as adding, editing, deleting, and searching data are implemented consistently across all sections, providing a solid foundation for managing business operations. I intentionally included a centralized main menu and modular design to enhance usability and streamline navigation, ensuring the system aligns with the project's primary goals.

However, I believe there are areas for improvement that could significantly enhance the system's overall functionality and user experience. The prototype currently lacks validation, which I plan to implement in future iterations to prevent invalid or incomplete data from being entered into the database. Additionally, I intentionally kept the interface design simple for the prototype phase, but I plan to improve it with better colour schemes, Mulligan Motor's logo, and more functions in future versions to make it more visually appealing and user-friendly. Features like advanced search filters, and report generation are also absent but are features I plan to include as the system

evolves. Furthermore, developing this prototype has allowed to understand its user requirements better and further refine my vision for the solution. I feel extra attributes such as ‘Vehicle Condition’ or ‘Date Joined’ could enhance the user experience. Simple improvements such as a log off button and exit form buttons were also overlooked in this prototype.

Overall, I believe this prototype is a strong starting point that successfully fulfils the essential requirements of Mulligan Motors. By intentionally focusing on the foundational aspects in this phase, I plan to build upon this solid base with usability, validation, and feature enhancements in future development, ensuring the system becomes a comprehensive and robust management tool that meets all the company’s needs.